

Capslock

25-26J-043

Project Proposal Report

Kulatunga K K M D
BSc (Hons) in Information Technology
Specialising in Cyber Security

CSE Department
IT Faculty
Sri Lanka Institute Information Technology

August 2025

Declaration

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Signature:

Date:

Signature of the Supervisor:



Date: 29/05/2025

Signature of the Co-Supervisor



Date: 29/05/2025

ABSTRACT

The accelerated delivery of cloud-native applications and services has been made possible by the increased usage of Kubernetes. However, it has brought to light important issues around the consistent enforcement of security policies across different environments. Existing solutions like ArgoCD, FluxCD, and Spinnaker, which are part of the DevOps and GitOps ecosystems, are capable of automating deployments. However, they do not enforce policies across different environments, lack ICAP services integration, and do not conduct risk-aware validation in pre-production and production environments. The inability to enforce policies consistently leads to configuration drift, fragmented security, and unmonitored vulnerabilities in the production environment.

This proposal is focused on the CAPSLock project and aims to address the gaps stated above with the introduction of the Multi-Environment Deployment System (MEDS). With MEDS, Integrated Continuous Auditing Policy (ICAP) and Risk-based Validation policies are embedded directly into the deployment pipeline, automating security enforcement within the pipeline. The described system has some important features such as ICAP integration, environment-aware orchestration, active environment risk validation, security policy versioning with rollback, and cross-environment visibility and audit logs.

By combining security-first orchestration with GitOps principles, MEDS enables the first Kubernetes-native , self-healing security framework.

Keywords — Kubernetes, GitOps, ICAP, Multi-Environment Deployment, Security Orchestration, Policy Management, Risk-Based Validation, DevSecOps

TABLE OF CONTENTS

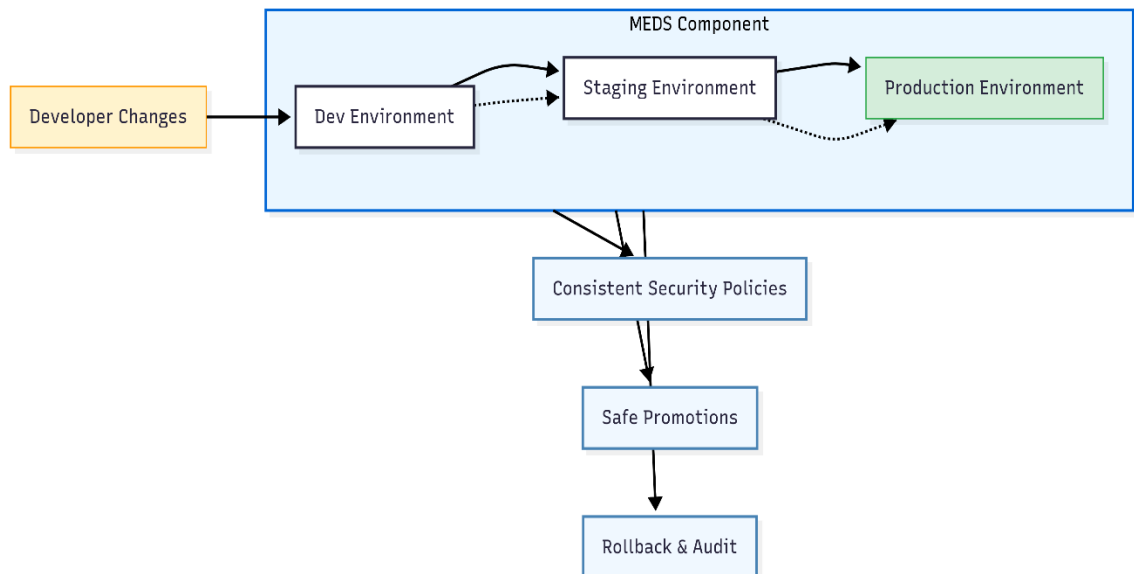
Declaration	Error! Bookmark not defined.
Abstract	3
1. Introduction	5
2. Background & Literature survey	6
3. Research Gap	8
4. Research Problem.....	11
5. Objectives.....	12
6. Methodology	13
7. Project Requirements	16
8. Budget and Budget Justification	18
9. References	20

1. INTRODUCTION

The CAPSLock research project addresses an age-old gap in container security; the inability to bridge the Internet Content Adaptation Protocol (ICAP) services with the ever-evolving Kubernetes-native orchestration. While there has been an industry-wide focus on bolstering containers, there still exists a critical structural void. Proxy-based security solutions, for example, are historically notorious for their inability to evolve with the fast-paced, ever-changing lifecycles of Kubernetes pods, not to mention the vendor lock-in proprietary integration traps.

The core of CAPSLock is the Multi-Environment Deployment System (MEDS), a solution specifically created to tackle the CIS benchmarks policies for the fourth research gap, environment inconsistency. MEDS goes beyond a mere deployment pipeline; it is a security policy orchestration engine with embedded multifactor security in the release lifecycle.

MEDS diverges from conventional pipeline logic, where velocity is paramount, instead treating security as an integral element. In MEDS, each step of the flow from development to staging and production incorporates layers of ICAP security checks and environment context-sensitive validation; thus, fostering a narrative where security is foundational. As a result, the platform ensures that applications not only traverse the stages quickly, but securely.



2. BACKGROUND & LITERATURE SURVEY

2.1 Containerization and Kubernetes Security

Portability, scalability, and resource efficiency have improved with the rapid uptake of containerization of applications. The automating of scaling, scheduling, and load balancing has made the use of containerized applications easier, and as a result, Kubernetes has become the industry standard container orchestrator [1]. However, containerization's shared-kernel architecture has unique security concerns when compared to virtual machines. For instance, one container's vulnerabilities can compromise other containers on the same host [2]. The distributed nature of Kubernetes clusters, as well as the ephemeral nature of pods, makes continuous monitoring and policy enforcement very difficult. Because of this, traditional security frameworks, which are designed for static systems, do not work well in dynamic orchestrated environments [3].

2.2 Internet Content Adaptation Protocol (ICAP) in Security Context

The Internet Content Adaptation Protocol (ICAP) (RFC 3507) focuses on proxy servers with content scanning functionalities, antivirus services, and content transformation services. While ICAP has been effective in traditional web security, its use in adoption in containerized environments is very limited. Current approaches rely on persistent proxy connections, which stand in opposition to Kubernetes' ephemeral pod lifecycles along with its dynamic service discovery [4]. In addition, current implementations of ICAP remain unintegrated with the orchestration of Kubernetes, instead depending on proprietary Application Programming Interfaces (APIs) which leads to vendor lock-in and fragmented architectures. [5]. This lack of standardization has resulted in unaddressed gaps in automated, standard, Kubernetes-native ICAP orchestration.

2.3 Multi-Cloud and Multi-Environment Deployment Challenges

To minimize costs, avoid provider lock-in, improve resilience, and improve flexibility, organizations deploy applications over a multi-cloud or hybrid environment [6]. Despite its many advantages, multi-cluster deployments come with a set of increased complexities in governance and compliance management, enforcement of policies, and there is still the cloud-specific detail which need to be considered abstraction [7]. The use of region-specific latency as a workload distribution is complicated due to region-based latencies [8]. Even more profound is the inconsistency of enforcement of policies over different clusters. The ever-evolving issues of environment drift and the declining of the overall security posture arise due to the configuration of agile development environments which use

permissive policies and agile-based development environments which use strict policies in production clusters [9].

2.4 Security Misconfigurations and Policy Enforcement Gaps

Research has emphasized common misconfigurations, such as too much access control, absent network policies, and insecure defaults within Kubernetes manifests [10]. These misconfigurations differ with the context, making the consistency of security enforcement from development to production a persistent challenge. The Square Compute Security team tackled this problem with the policy promotion lifecycle—Monitor → Warn → Enforce facilitating the evolutionary advancement of policies across environments within a safe operational framework [11]. This work highlights the necessity for policies that are hybrid, immutable, and static-dynamic, enabling the policies to shift gradually while preserving the ability to track changes and log-observe changes, principles of the MEDS system.

2.5 GitOps and Deployment Automation

The emergence of GitOps has changed how infrastructure and applications are managed by considering Git repositories as the single source of truth for system states. Solutions such as ArgoCD and Flux perform automated reconciliation of actual and desired states defined in version control, maintaining reproducibility, traceability, and rollback capabilities [12]. Reduction of configuration drift and human error in multi-cluster deployments has been one of the successes of GitOps [13]. Still, current GitOps workflows are centered around application deployment and do not incorporate enforcement of application policies, orchestration of workflows, or validation of governed workflows across multiple levels or environments [14]. MEDS builds upon these principles by incorporating validation and enforcement of evolving security policies using ICAP into GitOps workflows.

3. RESEARCH GAP

Despite rapid advancements in container security and DevSecOps practices, several critical shortcomings remain unaddressed in both academic research and industry solutions. A careful review of existing literature and tools reveals the following key gaps that form the foundation for this project:

3.1 ICAP Integration Deficits

The current Internet Content Adaptation Protocol (ICAP) use cases implement active content filtering for static proxy systems. These use cases completely ignore the interplay with Kubernetes' ephemeral pod lifecycles which may start and stop containerized applications within seconds [1]. This results in severe handshaking issues, scanning overhead, and inconsistent operations. Moreover, how existing ICAP solutions are implemented makes them highly dependent on proprietary application programming interfaces, aggravating vendor lock-in and reducing the use of different Kubernetes domains [2]. There has been little to no research on developing a Kubernetes native ICAP orchestration model, which leaves a very large gap in the standardization of container-aware content security.

3.2 Fragmented Security Toolchains

The Kubernetes security ecosystem is characterized by tool fragmentation. Organizations rely on a patchwork of scanners, admission controllers, service meshes, and monitoring tools. While each tool addresses a specific concern, their lack of integration results in:

- Overlapping or conflicting policies.
- Increased operational complexity.
- Gaps in coverage where threats may bypass security checks.

Academic studies confirm that this siloed approach leads to inefficiency and misconfigurations [3], yet few frameworks attempt to unify these tools under a coherent orchestration layer.

3.3 Multi-Environment Policy Inconsistency

Perhaps the most pressing research gap lies in inconsistent security policy enforcement across development, staging, and production environments. Studies highlight that:

- Development clusters often run with relaxed policies to enable rapid iteration.
- Production clusters enforce strict compliance controls.

- Staging environments frequently lack parity with production, creating a “false sense of security.”

This disparity, known as environment drift results in security gaps, misconfigurations, and increased risk of vulnerabilities propagating into production [4]. Although GitOps and CI/CD pipelines reduce configuration drift, no existing framework addresses the evolution of security policies across environments [5].

3.4 Lack of Risk-Aware Deployment Validation

Current CI/CD pipelines primarily validate functional correctness (e.g., passing tests, successful builds) but lack mechanisms for security-aware and risk-based validation. A deployment may be promoted to production if tests pass, even if it introduces policy violations, ICAP coverage gaps, or compliance regressions [6]. Research into risk-informed promotion pipelines is still nascent, leaving organizations without intelligent safeguards to predict and prevent security failures before they reach production.

Identified Research Gap

To summarize, the gaps can be distilled into three core areas:

1. Absence of a Kubernetes-native ICAP integration framework for handling ephemeral workloads [1][2].
2. Siloed security tools that lack unified orchestration and policy coherence [3].
3. No standardized approach for multi-environment policy evolution and risk-aware deployment validation [4][5][6].

3.5 Comparative Analysis: MEDS vs Existing Solutions

Current tools such as ArgoCD/FluxCD, Spinnaker, and Jenkins X provide strong deployment automation but do not address the key challenges of multi-environment policy consistency, ICAP integration, and risk-aware validation [1][2]. MEDS introduces these capabilities as part of its design, filling a critical gap in Kubernetes-native security orchestration.

Feature Comparison

Feature	GitOps (ArgoCD/FluxCD)	Helm + CI/CD	MEDS (Proposed)
Multi-Env Policy Consistency	Manual, inconsistent[3]	Per-environment	USLO framework
ICAP Integration	None[4]	Manual scripts	Native support

Risk-Based Validation	None	Basic health checks	Predictive validation[5]
Automated Rollback	Git revert only	Manual steps	SLO-triggered
Policy Versioning	Not supported	Not supported	Built-in versioning
Audit & Compliance	Git history only	Limited	Immutable audit log[6]

Existing Tools vs MEDS

- **ArgoCD / FluxCD** – Excellent for declarative GitOps deployments, but lack cross-environment policy orchestration and risk validation [3]. MEDS uses them as executors while adding a security orchestration layer.
- **Spinnaker** – Mature for multi-cloud pipelines but heavy and not security-focused [4]. MEDS offers a lightweight, Kubernetes-native alternative with integrated ICAP.
- **Jenkins X** – Early GitOps-based CI/CD tool, now deprecated and without policy support [2]. MEDS provides a modern, security-first architecture with policy evolution and rollback.

Novelty of MEDS

Compared to existing solutions, MEDS contributes three unique innovations:

1. **USLO Framework** – Unified security policy lifecycle across dev, staging, and prod [5].
2. **Risk-Based Validation** – Intelligent promotion gates analyzing compliance, configuration drift, and ICAP coverage.
3. **Integrated Rollback & Auditability** – Automatic rollback, versioned policies, and immutable audit logs for compliance reporting [6].

This research directly addresses these gaps through the Multi-Environment Deployment System. By embedding Unified Security Lifecycle Orchestration (USLO) and Risk-Based Deployment Validation into the CI/CD process, MEDS aims to provide the first holistic framework that unifies application promotion with consistent, adaptive, and security-aware policy enforcement.

4. RESEARCH PROBLEM

Research Problem Statement:

“In what ways may a Kubernetes-native deployment system be created to manage the cohesive application and policy promotion across various environments, incorporating ICAP-based security checks alongside risk-aware mechanisms for ongoing risk mitigation and continuous security guarantee throughout the software lifecycle?”

Despite significant progress in container orchestration and DevOps automation, multi-environment security consistency remains an unresolved challenge. Existing solutions such as GitOps tools (ArgoCD, FluxCD), Helm-based CI/CD pipelines, and Spinnaker pipelines automate deployments effectively but lack the ability to manage security policies, integrate ICAP services, or validate risks during environment promotions [1][2][3].

This creates three interrelated problems:

1. **ICAP Integration Deficit** – Traditional ICAP solutions cannot adapt to Kubernetes’ ephemeral workloads, resulting in incomplete content security coverage across environments [4].
2. **Fragmented Toolchains** – Current deployments rely on siloed tools for scanning, admission control, and monitoring, which increases operational complexity and leaves gaps in enforcement [3].
3. **Inconsistent Policy Evolution** – Security policies often diverge between dev, staging, and production, leading to environment drift, misconfigurations, and compliance risks [5][6].

As a result, application promotions are often judged only on functional correctness (tests, builds) without risk-aware validation of security posture. Misconfigurations and weak policies can therefore propagate from development into production, undermining trust in the entire deployment pipeline.

5. OBJECTIVES

5.1 Main Objective

To design and implement a Multi-Environment Deployment System that provides unified orchestration of applications and security policies across development, staging, and production Kubernetes environments, embedding ICAP-based validation [1], risk-aware promotion [2], and policy lifecycle management [3][4] to ensure consistent security assurance throughout the deployment pipeline.

5.2 Specific Objectives

1. Develop Multi-Cluster Deployment Support

- Enable seamless deployment and coordination of CAPSLock services (ICAP operator, policy engine) across multiple Kubernetes clusters [5].

2. Design an Environment Promotion Pipeline (Dev → Staging → Prod)

- Automate the controlled promotion of applications and policies between environments with integrated quality and security gates [6].

3. Implement Policy Migration and Rollback Mechanisms

- Provide versioned security policies that can evolve across environments, with rollback to last known-good states in case of misconfiguration or failure [3][7].

4. Integrate Risk-Based Deployment Validation

- Introduce predictive validation to detect configuration drift, compliance gaps, or weak ICAP coverage before promotions are approved [2][6].

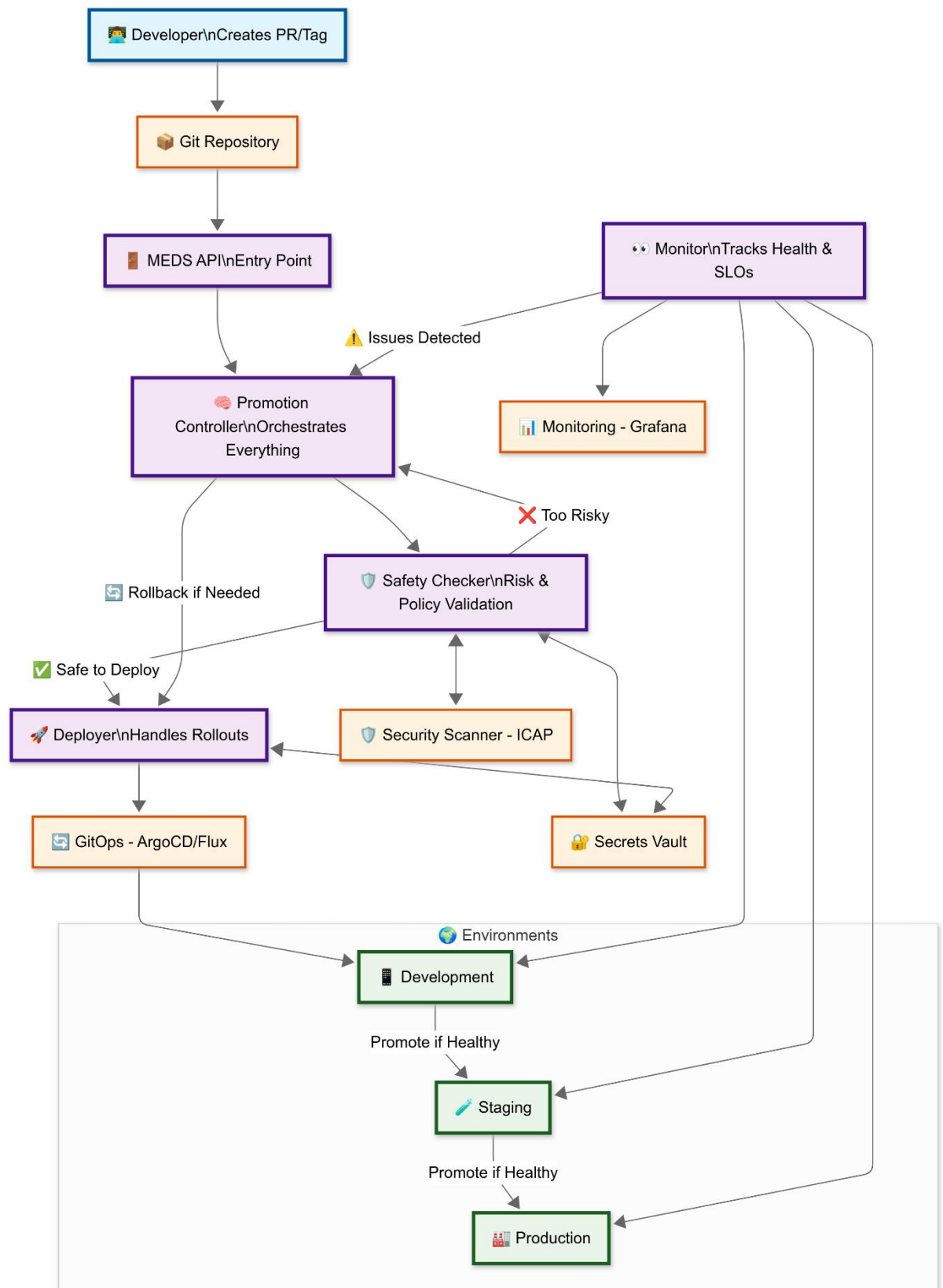
5. Build a Unified Management Interface

- Deliver a single-pane dashboard for cross-environment operations, including environment status, promotion controls, and immutable audit logs [8].

6. Conduct End-to-End Testing and Performance Validation

- Evaluate MEDS using realistic workloads, verifying functional correctness, rollback performance, policy enforcement, and scalability under stress [9].

6. METHODOLOGY



The proposed methodology for the Multi-Environment Deployment System combines principles of GitOps, CI/CD, and policy-as-code with Kubernetes-native ICAP integration. The design follows an iterative build–test–validate cycle, ensuring both functional correctness and security assurance.

6.1 System Architecture Overview

MEDS operates as a policy-driven orchestration layer sitting above existing GitOps and CI/CD executors (e.g., ArgoCD, FluxCD, or Jenkins). The architecture consists of:

- **Control Plane (MEDS Core):** Manages environment promotion logic, risk validation, and policy versioning.
- **Git Repositories:** Serve as the single source of truth for application manifests, Helm/Kustomize overlays, and environment-specific policy definitions [1].
- **Environment Clusters:** Three Kubernetes clusters (development, staging, production) where CAPSLock services (ICAP operator, policy engine) are deployed [2].
- **Policy Manager:** Applies and evolves security policies across environments in Audit or Enforce mode, supporting rollback to prior versions [3][4].
- **Unified Dashboard:** Provides visibility into environment status, pending promotions, audit logs, and risk assessment metrics [5].

This architecture ensures MEDS augments existing deployment tools with multi-environment policy orchestration and risk-aware validation, filling the gaps identified in current solutions [6][7].

6.2 Deployment Workflow

The MEDS deployment workflow proceeds as follows:

1. **Code Commit and Build:** Developers push code or configuration changes to the repository. CI pipelines build container images and run unit tests [1].
2. **Dev Deployment:** GitOps agents or CI/CD executors automatically deploy changes to the development cluster. Policies are applied in Audit mode, allowing observation without disruption [3].
3. **Validation and Promotion:** MEDS evaluates test outcomes, vulnerability scans, and risk metrics. If successful, changes are queued for promotion to

staging. Promotion rules may be automatic (for low-risk updates) or manual (requiring approval for sensitive updates) [6].

4. **Staging Deployment:** In staging, policies move closer to Enforce mode, and integration tests validate system behavior under production-like conditions [3][8].
5. **Risk-Based Validation:** Before production promotion, MEDS applies predictive validation, analyzing configuration drift, ICAP coverage, and compliance posture. If risks are detected, MEDS halts promotion until resolved [2].
6. **Production Deployment:** Approved changes are deployed to the production cluster. Rollout strategies such as blue-green or canary deployments may be used to minimize disruption [7].
7. **Rollback and Policy Versioning:** If an issue arises, MEDS can trigger automated rollback to previous versions for both applications and policies. Immutable logs ensure accountability [4][5].

6.3 Policy-Oriented Design

Unlike traditional pipelines, MEDS treats policies as first-class deployment artifacts:

- **Environment-Aware Policies:** Relaxed in development, stricter in production.
- **Promotion-Ready Policies:** Introduced in Audit mode, gradually evolved into Enforce mode [3].
- **Rollback-Capable Policies:** Each change versioned in Git, enabling safe reversion [4].

This Unified Security Lifecycle Orchestration (USLO) model ensures that policy enforcement matures as applications move from dev → staging → prod, eliminating inconsistencies across environments [3][6].

7. PROJECT REQUIREMENTS

7.1 Functional Requirements

- **Environment Promotion Pipeline:** Automate deployments across Dev → Staging → Prod with enforced order and approval gates [1][2].
- **Policy-as-Code Management:** Apply environment-aware policies (Audit → Enforce) with rollback support [3][4].
- **ICAP Integration:** Embed ICAP scanning directly into deployment workflows for risk validation [5].
- **Risk-Based Validation:** Evaluate configuration drift, compliance posture, and scan coverage before promotions [6].
- **Rollback & Versioning:** Maintain version-controlled rollback for both applications and policies with audit logging [2][4].
- **Multi-Cluster Support:** Enable consistent policy application across heterogeneous Kubernetes clusters [1][7].
- **Unified Dashboard:** Provide real-time visibility of environment status, policies, risks, and audit history [2][8].

7.2 Non-Functional Requirements

- **Security:** Enforce TLS for all communication, RBAC for access, and signed artifacts [3][5].
- **Reliability:** Ensure fault tolerance with retry mechanisms and a highly available MEDS control plane [2][7].
- **Performance:** Keep promotion latency under defined thresholds (<2 minutes for promotion checks; minimal ICAP overhead) [6].
- **Scalability:** Support multiple services and parallel promotions across environments [1][7].
- **Auditability:** Provide immutable logs and compliance-ready reporting (e.g., PCI-DSS, SOC2) [4][8].

7.3 Validation Approach

- **End-to-End Testing:** Verify full flow from Dev → Staging → Prod with consistent policy enforcement [2][6].
- **Security Tests:** Introduce misconfigurations to confirm ICAP validation and policy blocking [3][5].

- **Rollback Tests:** Simulate failed promotions and measure rollback speed [4][6].
- **Compliance Tests:** Validate audit logs and policy traceability against governance benchmarks [8].

8. BUDGET AND BUDGET JUSTIFICATION

The implementation of MEDS will primarily use open-source tools and student-accessible resources. Costs are minimized by leveraging free tiers, educational credits, and local infrastructure.

8.1 Estimated Budget

Resource	Details	Estimated Cost (USD)	Justification
Cloud Clusters (Student/Educational Credits)	GCP/AWS/Azure free credits; 2 small clusters (Staging & Prod), 2–3 nodes each for testing [1][2]	≈ \$100	Free credits reduce cost; covers extra usage during experiments.
Development Environment	Local Minikube/k3s/kind on personal/institute machines [3]	\$0	No extra cost; suitable for Dev environment.
CI/CD Infrastructure	GitHub Actions (student free tier), Jenkins self-hosted [4]	\$0	Free tiers sufficient for academic pipeline setup.
Domain + SSL (Optional)	Use free .github.io or university subdomain; Let's Encrypt SSL [5]	\$0	Avoids paid domain; secure access still possible.
Testing Tools & Data	Public vulnerable images, EICAR malware samples, open CVE datasets [6][7]	\$0	Freely available; no licensing costs.
Contingency	Small buffer for unforeseen cloud/network usage	\$30	To cover unexpected cluster hours or storage.

Total Estimated Cost: ≈ \$130 USD

8.2 Budget Justification

- **Cloud Infrastructure:** Minimal cost, as most can be covered by free student credits. The small buffer allows scaling if test cases exceed free quotas [1][2].
- **Development Environment:** Fully local, using Minikube/kind, avoiding cloud costs for the dev stage [3].
- **CI/CD:** Reliance on free/open-source CI/CD platforms ensures no recurring costs [4].
- **Domain/SSL:** Avoided paid domain registration by using free hosting options and Let's Encrypt SSL [5].
- **Testing Resources:** Freely available test datasets and images ensure realistic security validation without licensing fees [6][7].
- **Contingency:** A small budget allocation ensures experiments are not blocked by unexpected usage.

9. REFERENCES

- [1] B. Erdenebat, B. Bud, T. Batsuren, and T. Kozsik, “Multi-Project Multi-Environment Approach—An Enhancement to Existing DevOps and CI/CD Tools,” *Computers*, vol. 12, no. 12, Art. no. 254, Dec. 2023. doi: 10.3390/computers12120254.
- [2] V. Goyal, “Multi-Cluster CI/CD Pipelines Across Dev, Staging, Prod & DR Environments,” *Devtron Blog*, Jun. 2025. [Online]. Available: <https://devtron.ai/blog/multi-cluster-ci-cd-pipelines/>
- [3] A. Gogineni, “Multi-Cloud Deployment with Kubernetes: Challenges, Strategies, and Performance Optimization,” *Int. Scientific Journal of Engineering and Management*, vol. 1, no. 2, pp. 1–7, Sept. 2022. doi: 10.55041/ISJEM00036.
- [4] C. Luca, “GitOps in Multi-Cloud and Hybrid Cloud Environments,” *ResearchGate Preprint*, Dec. 2022. [Online]. Available: <https://www.researchgate.net/publication/388109376>
- [5] A. Rahman, S. I. S. Shamim, D. B. Bose, and R. Pandita, “Security Misconfigurations in Open Source Kubernetes Manifests: An Empirical Study,” *ACM Trans. Softw. Eng. Methodol.*, vol. 32, no. 1, Art. no. 3, Jan. 2023. doi: 10.1145/3571234.
- [6] M. A. Rahman, M. Hassanpour, T. Abdelzaher, and I. Gupta, “Kubernetes Scheduling: Taxonomy, Ongoing Issues and Challenges,” *ACM Computing Surveys*, vol. 55, no. 4, pp. 1–37, Jul. 2022. doi: 10.1145/3529444.
- [7] Square Compute Security, “Kube-Policies: Guardrails for Apps Running in Kubernetes (Part 1),” *Square Developer Blog*, Oct. 2023. [Online]. Available: <https://developer.squareup.com/blog/kube-policies-guardrails-for-apps-running-in-kubernetes/>
- [8] I. Elgendi, A. Mustapha, Q. Nasir, and A. Marka, “Container Security: Issues, Challenges, and the Road Ahead,” *IEEE Access*, vol. 7, pp. 52976–52996, Apr. 2019. doi: 10.1109/ACCESS.2019.2912540.
- [9] S. Sultan, I. Ahmad, and T. Dimitriou, “Container Security: Issues, Challenges, and the Road Ahead,” *IEEE Access*, vol. 7, pp. 52968–52975, Apr. 2019. doi: 10.1109/ACCESS.2019.2912413.
- [10] Devtron, “Environment-Aware CI/CD and Governance,” *Devtron Whitepaper*, 2025. [Online]. Available: <https://devtron.ai>